Assignment 3: CNN-LSTM, ViT, and Object Tracking

Instructions:

- Follow the instructions of each question carefully.
- Submit each section as a separate **jupyter notebook** that shows outputs of each stage.
- **If you cannot explain any part of the code, you will be given zero for that part of the assignment.**

Section 1: Action Recognition using CNN-LSTM on the UCF11 Dataset (100 marks)

The goal of this task is to implement an action recognition model using a combination of CNN and LSTM. You will train the model on the **UCF11 dataset**, which contains action videos categorized into 11 different classes.

Dataset:

1. Download UCF11 dataset from https://www.crcv.ucf.edu/data/UCF11_updated_mpg.rar. Unrar the file to generate a folder with the following folder structure:

UCF11

|---- walking

|---- volleyball_spiking

…

2. Use the provided **ucf11_helper.ipynb** with the following changes: a) change the path of `input_folder` to the folder generated in the previous step and b) create an `output_folder` and set its path in `output_folder`. The `output_folder` will contain the following CSV files after you run **ucf11_helper.ipynb**

- **train_map.csv**: Contains file paths and corresponding integer labels for training data.
- **val_map.csv**: Contains file paths and labels for validation data.
- **test_map.csv**: Contains file paths and labels for test data.
- Each CSV file will **two columns**:
    - `filepath` – The path to the video file.
    - `label` (0-10) – An integer representing the action category.

Tasks:

1. **Implement the UCF11 Dataset Loader**:
    - Read the CSV file.
    - Load each video file.
    - Extract **16 frames per video,** convert each frame to grayscale and resize to **64x64** pixels.
    - Normalize the pixel values and return the tensor.
2. **Implement the CNN-LSTM Model**:

- Use a **CNN-based feature extractor** ResNet-18 from torchvision.models. Modify it to take grayscale images. Freeze the CNN model. We will only train the LSTM.
- Extract frame-level features and pass them into an **LSTM** with 256 hidden dimensions and 1 layer (use nn.LSTM)
- Use the final LSTM hidden state for classification.

3. **Train the Model**:
   - Use **CrossEntropyLoss** as the loss function.
   - Use **Adam optimizer** with a learning rate of **0.001**.
   - Train for **5 epochs** and evaluate on the validation set after each epoch.
4. **Test the Model**:
   - Report the training loss curve and train and validation accuracy after every epoch in a graph.
   - Report the final test accuracy.

Section 2: Vision Transformer (100 marks)

You are tasked with implementing a Vision Transformer (ViT) from scratch and fine-tuning it on the CIFAR-10 dataset. Your implementation should include the following components:

1. **Multi-Head Self-Attention (MHSA):**
   - Implement the multi-head attention mechanism using query, key, and value projections.
   - Compute scaled dot-product attention.
   - Apply linear transformation to the concatenated attention output.
2. **Transformer Encoder Block:**
   - Utilize the MHSA module inside a Transformer block.
   - Implement layer normalization and a feed-forward MLP with 2 linear layers and GeLU activation.
   - Apply residual connections after attention and MLP layers.
3. **Vision Transformer (ViT) Architecture:**
   - Convert images into patches and embed them.
   - Add learnable class tokens and positional embeddings.
   - Stack 2 Transformer encoder blocks.
   - Use a classification head to predict CIFAR-10 classes.
4. **Training and Inference:**
   - Load the CIFAR-10 dataset and apply necessary transformations.
   - Train the ViT model using the cross-entropy loss.
   - Evaluate the model on the test dataset.

Tasks:

1. Implement the Multi-Head Self-Attention mechanism.
2. Implement a Transformer Encoder Block **without using** `nn.TransformerEncoderLayer`.
3. Implement a Vision Transformer (ViT) for CIFAR-10 classification (dataset available in torchvision.datasets).
4. Train the ViT model for 25 epochs and plot the training loss per epoch. Finally, report the test accuracy.

Section 3: Multi-Object Tracking System (100 marks)

Design and implement a tracking-by-detection multi-object tracking system using:

- Object detector
- Kalman filter motion model
- IoU-based Hungarian matching
- Track lifecycle management

1. Object Detection

Use YOLO v8

```
from ultralytics import YOLO
model = YOLO("yolov8n.pt")
```

Constraints:

- Detect only humans
- `Detection_Confidence_threshold` must be configurable
- Non-Max Suppression threshold must be configurable

2. Motion Model

Implement a **constant velocity Kalman filter**.

State vector: $[x, y, w, h, v_x, v_y]$

You must implement:

- Predict step
- Update step
- State transition logic
- Basic process noise

Use of external Kalman libraries is NOT permitted.

3. Data Association

You must implement:

- Intersection over Union (IoU)
- Cost matrix generation: Cost=1−IoU
- Hungarian algorithm (via scipy)

Matching rule: Reject matches if $IoU < \tau$ where IoU threshold τ is configurable.

4. Track Management

Each track must maintain:

- Unique track ID
- Age
- Number of hits

- Time since last update

Lifecycle rules:

- Unmatched detection → create new track
- Unmatched track → predict only
- Track removed if missed for `max_age` frames
- Track confirmed after `max_age`

All lifecycle thresholds must be configurable.

5. Train–Test Protocol

Phase 1: Training (tracking_train.avi)

- Run tracker on `tracking_train.avi`
- Tune:
  - IoU threshold τ
  - `max_age`
  - `min_hits`
  - `Detection_Confidence_threshold`
- Analyze tracking failures
- Select final configuration for the 4 parameters above

Phase 2: Testing (tracking_test.avi)

Rules:

- No further tuning allowed
- No code modifications allowed
- Run with final configuration obtained above

6. Output requirements

- `train_output.avi`
- `test_output.avi`

Each frame in the video must display:

- Bounding box
- Track ID